Multilingual Clinical Dialogue System with Lifestyle Assessment

Submitted in partial fulfillment of the requirements

For the degree of

BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

By

Nidhi Sinha (39)

Sarvesh Parab (40)

Under Guidance of
Prof. Deepti Chandran

Department of Computer Engineering



Smt. Indira Gandhi College of Engineering

Autonomous Institute, Affiliated to University of Mumbai
(2026-2027)

INDEX

S. No.	Chapter / Section	Page No.
1	**Abstract**	**3**
2	**Objective and Scope**	**4**
2.1	Objectives	4
2.2	Scope of the System	4
2.3	Out of Scope	4
3	**System Analysis**	**5**
3.1	Existing System	5
3.2	Limitations of the Existing System	5
3.3	Proposed System	5
3.4	Feasibility Study	5
3.4.1	Technical Feasibility	5
3.4.2	Economic Feasibility	5
3.4.3	Operational Feasibility	6
3.5	Functional Requirements	6
3.6	Non-Functional Requirements	6
3.7	Hardware and Software Requirements	6
3.7.1	Development and Server Requirements	6
3.7.2	Client Requirements	7
4	**System Design**	**8**
4.1	System Architecture	8
4.2	Module Description	8
4.2.1	Authentication and Consent Module	8
4.2.2	Speech and Language Module	9
4.2.3	Conversation Engine	9
4.2.4	Scoring and Risk Module	9
4.2.5	Medical NLP Module	9
4.2.6	Recommendation Module	9
4.2.7	Reporting Module	9
4.3	Conversation Workflow	9
4.4	Database Design	10
4.5	Wellness Score Design	10
4.6	Risk Rule Design	11
4.7	Safety Design	11
5	**Implementation**	**12**
5.1	Technology Stack	12
5.2	Implementation of Key Modules	12
5.2.1	Lifestyle Slot Extraction	12
5.2.2	Wellness Score Calculation	13
5.2.3	Risk Rule Evaluation	13
5.2.4	Emergency Escalation	14
5.2.5	Guideline Retrieval	14
5.2.6	Doctor Override and Re-scoring	15
5.3	Output Screenshots	15
6	**Results and Conclusion**	**18**
6.1	Testing and Verification	18
6.2	Sample Case Study	18
6.3	Observations	18

6.4	Conclusion	19
7	**Future Scope**	**20**
7.1	Wearable Device Integration	20
7.2	Longitudinal Progress Tracking	20
7.3	AI-Assisted Nutrition Planning	20
7.4	Mental Health Assessment Module	20
7.5	Medication Adherence Reminders	20
7.6	Voice Emotion Analysis	20
7.7	Hospital Information System Integration	20
7.8	Expanded Language Coverage	20
7.9	Clinical Validation Study	20
8	**References**	**21**

LIST OF FIGURES

Fig. No.	Title	Page No.
Fig 1	System Architecture	8
Fig 2	Workflow Diagram	10
Fig 5.1	Login screen	15
Fig 5.2	Doctor's patient queue	15
Fig 5.3	Doctor-led consultation recording, English	16
Fig 5.4	Doctor-led consultation recording, Hindi	16
Fig 5.5	Doctor's clinical detail view	16
Fig 5.6	Risk flags and guideline-grounded recommendations	17

1. ABSTRACT

Lifestyle habits such as sleep duration, physical activity, diet quality, hydration, stress and tobacco or alcohol use are among the strongest modifiable contributors to chronic disease. In routine outpatient practice, however, these habits are rarely recorded in any structured form. A doctor in a busy Indian clinic typically has only a few minutes per patient, and that time is spent on the presenting complaint. Where the patient and the doctor do not share a fluent common language, the gap widens further, because a detailed lifestyle history is exactly the kind of conversation that language barriers disrupt.

This project presents the Multilingual Clinical Dialogue System, a web application that collects a structured lifestyle history through a natural conversation in the patient's own language, converts it into clinical data, and presents it to the doctor as a reviewable summary. The patient speaks or types in one of nine supported Indian languages. The system transcribes the speech, translates it internally to English, and an assistant asks about eight lifestyle areas one at a time, adapting its questions to what the patient has already said.

A deliberate design decision underpins the whole system: the language model is used only to understand what the patient said, never to make a clinical judgement. Every extracted value is stored with the patient's own words and a confidence level. The Lifestyle Wellness Score, the risk flags and the body mass index are computed by a fixed weight table written in plain Python, so the same inputs always produce the same output and the calculation can be audited line by line. Health recommendations are not generated freely either; they are retrieved from World Health Organization physical activity guidelines and ICMR-NIN dietary guidelines using a vector database, and each recommendation carries the source document and page number it came from.

The system includes an emergency override. If the patient reports chest pain, severe breathlessness, stroke signs or similar red-flag symptoms, the lifestyle questionnaire stops immediately, the consultation is marked as escalated, and the patient is directed to seek in-person medical care. No score and no lifestyle advice are produced in that case.

The application was built with Django REST Framework and a Next.js frontend, with SQLite for storage and ChromaDB for guideline retrieval. It supports both patient-led consultations and doctor-led in-clinic recording, where the doctor holds the device while the patient speaks. The doctor can correct any extracted value, at which point the score is recalculated, and can download a printable clinical report that renders correctly in Devanagari and six other Indic scripts. The finished system passes 300 automated tests covering the scoring table, the risk rules, the escalation path and the access-control boundaries between patients and doctors.

The system is a screening and documentation aid. It does not diagnose, does not name or prescribe medication, and defers every clinical decision to the doctor.

2. OBJECTIVE AND SCOPE

2.1 Objectives

The project was undertaken with the following objectives:

- To collect a complete lifestyle history from a patient through natural conversation, without adding to the doctor's consultation time.
- To allow that conversation to take place in the patient's own language, so that patients who are not comfortable in English are not excluded or under-recorded.
- To convert free-form spoken answers into structured, machine-readable clinical fields, each traceable back to the words the patient actually used.
- To compute a Lifestyle Wellness Score using a fixed, transparent and reproducible rule set rather than a language model, so that the number can be defended and audited.
- To detect lifestyle-related health risks, including composite risks such as possible hypertension risk, and to state the reason behind every flag raised.
- To generate health recommendations that are grounded in published clinical guidelines and that cite their source, rather than being invented by the model.
- To detect medical emergencies during the conversation and redirect the patient to immediate in-person care instead of continuing a lifestyle questionnaire.
- To present all of the above to the doctor in a single reviewable screen, where every value can be corrected before it counts, and to produce a printable clinical report.

2.2 Scope of the System

The system covers the complete path from a patient's first utterance to a signed-off clinical record. The following functionality is within scope:

- Patient registration, consent capture and login, with separate roles for patients and doctors.
- Voice input through the browser microphone and text input as a fallback, in nine languages: English, Hindi, Marathi, Tamil, Telugu, Bengali, Kannada, Gujarati and Malayalam.
- Automatic speech recognition and internal translation to English for processing, while the patient continues to see and hear their own language.
- A slot-filling conversation covering eight lifestyle areas: sleep, exercise, diet, water intake, stress, sitting hours, smoking and alcohol.
- Extraction of symptoms from the transcript using a biomedical named entity recognition model.
- Deterministic wellness scoring, risk-flag detection and body mass index calculation using Asian-Indian cutoffs.
- Retrieval-augmented generation of health recommendations from guideline documents, with page-level citations.
- A doctor dashboard with a patient queue, a full clinical detail view, inline correction of any extracted value with live re-scoring, doctor's notes, and a review sign-off.
- Doctor-led consultation mode, in which the doctor starts a consultation for a patient and records it during the in-clinic visit.
- PDF clinical report generation with embedded fonts for Indic scripts.

2.3 Out of Scope

The following are deliberately outside the scope of this project and are noted here so that the system's boundaries are clear:

- The system does not produce a diagnosis. It identifies lifestyle risk factors for the doctor to interpret.
- The system does not name, recommend or adjust any medication.
- The system does not replace any part of the physical examination or laboratory investigation.
- The system has not been clinically validated. The wellness score is a scoring convention constructed from published guideline thresholds, not a validated clinical instrument.
- The system does not integrate with hospital management or electronic medical record software in the current version.

3. SYSTEM ANALYSIS

3.1 Existing System

In current outpatient practice, lifestyle history is collected in one of three ways. Most commonly it is not collected at all beyond a passing question, because consultation time is short. Where it is collected, it is either written as free text in the doctor's own notes, or captured on a paper intake form that the patient fills in before the consultation.

A small number of hospitals use digital intake forms, usually a fixed questionnaire presented on a tablet. These are almost always in English, occasionally with a single regional language option, and they present all questions at once regardless of what the patient has already said.

3.2 Limitations of the Existing System

Each of the existing approaches has specific shortcomings that this project addresses:

- **Information loss.** Lifestyle details recorded as free text inside a consultation note cannot be searched, compared across visits, or used to compute anything. The information exists but is not usable data.
- **Time pressure.** Asking eight lifestyle questions properly takes several minutes. In a clinic seeing forty or more patients a day, that time is not available, so the questions are dropped.
- **Language exclusion.** An English-language intake form is filled in poorly or not at all by patients who are not comfortable reading English, which is a substantial proportion of patients in Indian primary care.
- **Recall failure.** Patients frequently forget to mention habits that are clinically relevant, particularly when nobody asks about them specifically.
- **No risk synthesis.** Even where all the individual facts are recorded, nothing combines them. A patient with a sedentary routine, an unhealthy diet and high stress presents three separate notes rather than one flag saying that these together indicate a possible hypertension risk.
- **Rigid questioning.** A fixed digital form asks about sleep even when the patient has already described their sleep in their opening complaint, which wastes the patient's patience and reduces the quality of the remaining answers.

3.3 Proposed System

The proposed system replaces the intake form with a conversation. The patient speaks naturally, in their own language, and an assistant asks follow-up questions one at a time, skipping what has already been answered and adapting its wording to what the patient said. The conversation is driven by a deterministic state machine: the assistant decides how to phrase a question, but which area is asked next, when the questionnaire ends, and whether the consultation escalates are decided by ordinary program logic.

The answers are extracted into a fixed set of structured fields. Every field carries the patient's own phrasing as evidence and a confidence value. Where confidence is low, the system asks one clarifying question and then moves on rather than guessing. Where fewer than four of the eight areas end up answered, the system deliberately produces no score at all, because a precise-looking number computed from two answers would mislead the doctor.

The doctor receives the result as a structured summary and retains full control over it. Any extracted value can be corrected in place, and the score, the risk flags and the report update immediately to reflect the correction.

3.4 Feasibility Study

3.4.1 Technical Feasibility

All components of the system are built on mature, freely available technology. The backend uses Django and Django REST Framework, the frontend uses Next.js, and the database is SQLite, all of which are widely documented and stable. Speech recognition and language understanding are accessed through a hosted inference API, which removes the need for local GPU hardware. Biomedical entity recognition uses scispaCy, which runs on an ordinary CPU. Guideline retrieval uses ChromaDB with a small embedding model that also runs on CPU. The complete application runs on a single low-cost virtual private server, which was verified during development.

3.4.2 Economic Feasibility

The project uses open-source software throughout, so there are no licence costs. The two guideline documents used for retrieval are published free of charge by the World Health Organization and the National Institute of Nutrition. The language and speech inference API used during development offers a free usage tier sufficient for

the volumes involved in a project of this scale. The only recurring cost in a production deployment would be server hosting, which is modest because the application has light resource requirements.

3.4.3 Operational Feasibility

From the patient's side the system requires nothing more than speaking into a phone or a clinic tablet in their own language, which requires no training. From the doctor's side, the system presents a single screen per patient and does not change the existing consultation workflow; it adds information rather than adding steps. The doctor-led mode exists specifically so that the system fits the common Indian clinic pattern in which the patient does not own a smartphone or is not comfortable using one.

3.5 Functional Requirements

ID	Requirement
FR-1	The system shall allow a patient to register, record consent, and log in.
FR-2	The system shall allow a patient to start a consultation and send messages by voice or text.
FR-3	The system shall transcribe voice input and translate it internally to English for processing.
FR-4	The system shall ask about eight lifestyle areas, one at a time, in the patient's chosen language.
FR-5	The system shall extract structured values from patient answers along with a source quote and confidence.
FR-6	The system shall detect emergency symptoms and escalate the consultation without asking further lifestyle questions.
FR-7	The system shall compute a Lifestyle Wellness Score from a fixed weight table.
FR-8	The system shall detect lifestyle-related risk factors and state a reason for each.
FR-9	The system shall generate health recommendations retrieved from guideline documents with page citations.
FR-10	The system shall present a doctor with a queue of consultations, filterable and searchable.
FR-11	The system shall allow a doctor to correct any extracted value and recalculate the score.
FR-12	The system shall allow a doctor to add notes and mark a consultation as reviewed.
FR-13	The system shall generate a downloadable PDF clinical report.
FR-14	The system shall allow a doctor to start and record a consultation on the patient's behalf during a clinic visit.

3.6 Non-Functional Requirements

Attribute	Requirement
Reproducibility	Identical lifestyle inputs must always produce an identical wellness score. No scoring step may depend on a language model.
Traceability	Every extracted value must retain the patient's own words and a confidence level, visible to the doctor.
Safety	Emergency symptoms must halt the questionnaire on the same turn they are detected, before any further question is asked.
Correctness of advice	No recommendation may cite a source that was not actually retrieved. Where nothing relevant is found, the system must say so and fall back to general guidance.
Language support	The patient-facing conversation and the printed report must render correctly in Devanagari and six other Indic scripts.
Security	Authentication uses JSON Web Tokens. A doctor may not access a consultation they were not assigned to, and a patient may access only their own.
Privacy	Patient consent must be recorded before any conversational data is collected.
Performance	A conversational turn, including transcription and extraction, must complete within a few seconds on an ordinary broadband connection.
Testability	Core logic must run offline in a deterministic mock mode so that the full test suite executes without network access.

3.7 Hardware and Software Requirements

3.7.1 Development and Server Requirements

Component	Specification
Processor	Intel Core i3 or equivalent, dual core, 2.0 GHz or higher
Memory	4 GB RAM minimum, 8 GB recommended
Storage	5 GB free disk space

Component	Specification
Operating System	Windows 10/11, Ubuntu 20.04 or later, or macOS
Runtime	Python 3.10 or later, Node.js 18 or later
Network	Broadband internet connection for the inference API

3.7.2 Client Requirements

Component	Specification
Device	Any smartphone, tablet or desktop computer
Browser	A current version of Chrome, Edge, Firefox or Safari
Microphone	Required for voice input; text input is available as a fallback
Connection	HTTPS is required for microphone access outside local development

4. SYSTEM DESIGN

4.1 System Architecture

The system follows a layered client-server architecture. The presentation layer is a Next.js single-page application that serves both the patient conversation interface and the doctor dashboard. It communicates with the application layer exclusively over a REST interface authenticated with JSON Web Tokens; it never contacts the language model directly.

The application layer is a Django REST Framework service containing four functional units. The conversation engine manages the dialogue state, decides which lifestyle area to ask about next, extracts structured values from the patient's answer and screens each message for emergency symptoms. The scoring and risk unit computes the wellness score, the risk flags and the body mass index in plain Python. The medical NLP unit extracts symptoms and diseases from the transcript. The recommender unit retrieves relevant guideline passages and composes recommendations from them.

The data layer holds a relational database for consultation records and a vector database for guideline chunks. The speech and language inference service is the only external dependency and sits outside the deployment boundary; everything else is self-hosted.

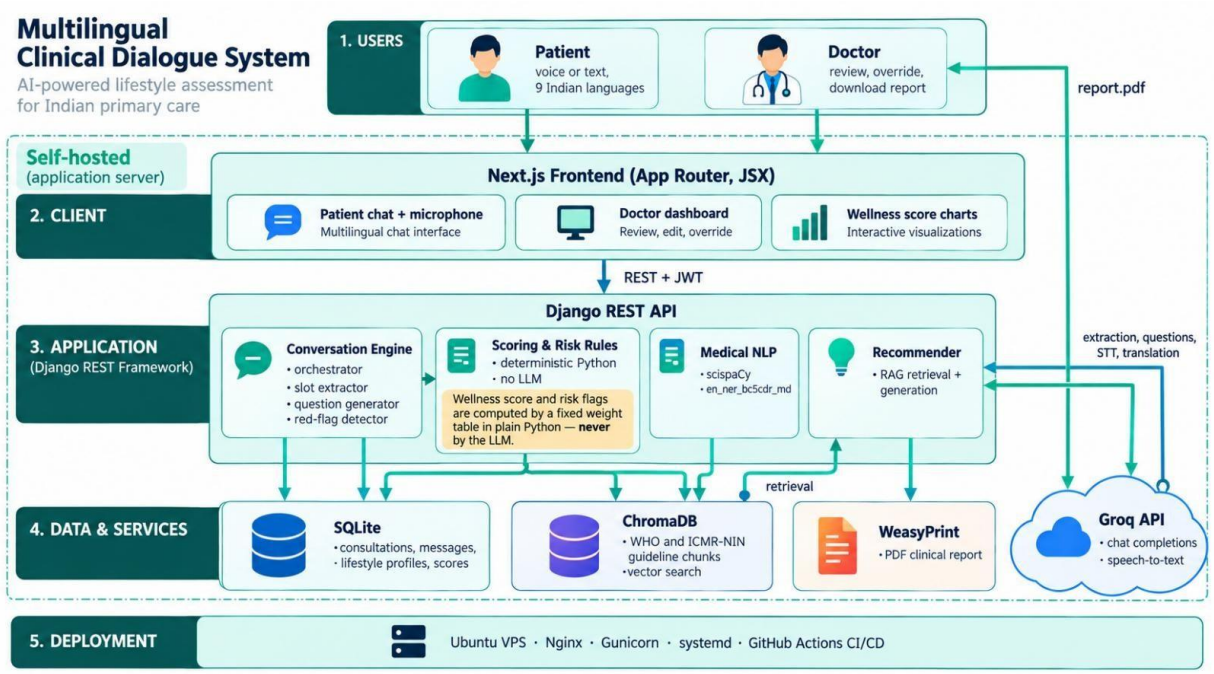


Fig 1 System Architecture

Layer	Components
Presentation	Next.js application: patient chat with microphone, doctor queue, doctor clinical detail view, wellness charts
Application	Django REST API: conversation engine, scoring and risk rules, medical NLP, recommender
Data	SQLite for consultations and clinical records; ChromaDB for guideline vector search
External Service	Hosted inference API for speech-to-text and chat completions
Deployment	Ubuntu server with Nginx, Gunicorn and systemd; continuous deployment through GitHub Actions

4.2 Module Description

4.2.1 Authentication and Consent Module

Maintains two user roles. Patients register themselves and must record explicit consent before any consultation can begin. Doctors are created administratively. The module enforces that a patient may access only their own consultations and that a doctor may access only consultations they started or were assigned. Where a doctor records a consultation on the patient's behalf, verbal consent is confirmed and stored separately, along with the identity of the doctor who recorded it.

4.2.2 Speech and Language Module

Receives an audio recording from the browser, transcribes it in the patient's language, and produces an English translation for internal processing. The original transcription is what the patient sees. Where the consultation language is already English, the translation step is skipped entirely to avoid an unnecessary API call. All external calls pass through a single service wrapper that handles retries, timeouts and a deterministic offline mode used for testing.

4.2.3 Conversation Engine

The core of the system. It maintains eight slots corresponding to the eight lifestyle areas, each in one of three states: pending, filled or skipped. For each patient message it performs, in order: an emergency symptom check, extraction of any lifestyle values present, persistence of those values, and then either generation of the next question or completion of the consultation. Extraction results below a confidence threshold trigger one clarifying question; if the answer is still unclear, the slot is marked skipped and the conversation moves on. A turn limit prevents an unbounded conversation.

4.2.4 Scoring and Risk Module

Computes the Lifestyle Wellness Score from a fixed weight table and evaluates ten risk rules. No part of this module calls a language model. Unanswered areas are excluded from the calculation and the remaining weights are rescaled, but if fewer than four areas are answered the module produces no score at all. Doctor corrections take precedence over extracted values throughout.

4.2.5 Medical NLP Module

Runs a biomedical named entity recognition pipeline over the English text of the consultation to identify symptoms and disease mentions, which are deduplicated against any symptoms the extraction step already captured. This module runs entirely locally on the CPU.

4.2.6 Recommendation Module

For each risk flag raised, the module constructs a query, retrieves the closest matching passages from the guideline vector store, and composes a short practical recommendation grounded strictly in those passages. A relevance threshold prevents unrelated passages from being used; where nothing relevant is found the module falls back to general guidance and labels it as such. Retrieval is filtered by the patient's age group so that an adult never receives guidance written for children. Any citation not present in the retrieved passages is discarded.

4.2.7 Reporting Module

Renders the complete clinical record as a printable PDF containing patient details, symptoms, the lifestyle table, the score with its breakdown, risk flags with their reasons, recommendations with sources, doctor's notes and a disclaimer. Fonts for Devanagari and six other Indic scripts are embedded in the document so that non-Latin text renders correctly without relying on the reader's system fonts.

4.3 Conversation Workflow

A complete consultation follows the sequence below. The loop between steps two and six is what distinguishes this system from a fixed questionnaire.

Step	Action	Outcome
1	Patient begins consultation	Consent verified; consultation record created
2	Patient speaks or types	Audio or text received in the patient's language
3	Transcribe and translate	Original text retained; English copy used internally
4	Emergency screening	If a red flag is found, escalate and stop; otherwise continue
5	Extract lifestyle values	Slots filled with values, evidence quotes and confidence
6	Check remaining slots	If any are unanswered, ask the next one and return to step 2
7	Run analysis	Score, risk flags and recommendations generated
8	Doctor review	Values corrected as needed; score recalculated
9	Sign off	Consultation marked reviewed; PDF report generated

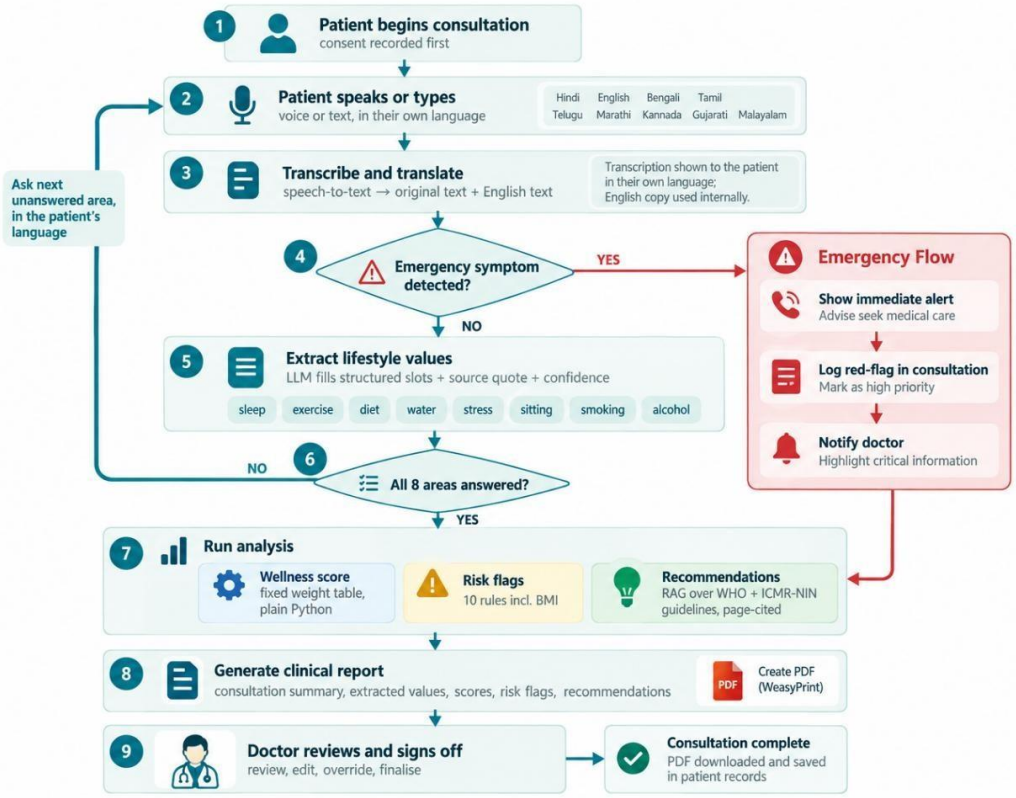


Fig 2 Workflow Diagram

4.4 Database Design

The database holds nine principal entities. The design keeps the raw conversation, the extracted structured data and the derived clinical output in separate tables, so that re-running the analysis never destroys the original record of what the patient said.

Entity	Principal Fields
User	id, username, password, role (patient or doctor)
PatientProfile	user, full name, age, sex, height, weight, preferred language, consent timestamp, consent recorded by
Consultation	id, patient, doctor, language, status, chief complaint (original and English), turn count, timestamps
Message	consultation, sender, original text, English text, language, audio file, target slot, created at
LifestyleProfile	consultation, sleep hours, exercise days and minutes, smoking status, alcohol frequency, diet pattern, water litres, stress level, sitting hours, evidence, doctor overrides
Symptom	consultation, name, duration, severity, source
RiskFlag	consultation, code, label, severity, reason
WellnessScore	consultation, total, band, breakdown
Recommendation	consultation, category, English text, patient-language text, sources, order
DoctorNote	consultation, doctor, text, created at

4.5 Wellness Score Design

The Lifestyle Wellness Score is the central quantitative output of the system, and its design is deliberately simple so that it can be explained and defended. Each of the eight lifestyle areas is assigned a weight, and each area's reported value is converted to a sub-score between zero and one against published guideline thresholds. The final score is the weighted sum, expressed out of one hundred.

Factor	Weight	Full marks (1.0)	Partial	Zero
Sleep	15	7-9 hrs/day	6-7 or 9-10 hrs	Below 5 or above 10 hrs
Exercise	15	150+ min/week	75-149 min	None
Diet	15	Balanced	Mixed or irregular	Frequent fast food or high sugar
Stress	15	Low	Moderate	High

Factor	Weight	Full marks (1.0)	Partial	Zero
Smoking	15	None	Occasional	Daily
Alcohol	10	None	Occasional or weekly	Daily
Sitting	10	Under 6 hrs/day	6-10 hrs	Above 10 hrs
Water	5	2-3.5 L/day	1-2 L or above 4 L	Below 1 L

The resulting total is mapped to one of four bands: 80 to 100 is Good, 60 to 79 is Fair, 40 to 59 is Needs attention, and below 40 is Poor.

Two design details deserve note. First, an unanswered area is excluded from the calculation rather than scored as zero, and the remaining weights are rescaled to total one hundred; but if fewer than four of the eight areas are answered, no score is produced at all, because rescaling two answers across the full scale produces a misleadingly confident number. Second, both ends of each scale are bounded. Water intake illustrates why: an open-ended rule would award full marks for five litres a day, which is not a healthy intake, so intake above four litres is scored down and flagged for the doctor's attention.

4.6 Risk Rule Design

Ten risk rules are evaluated after scoring. Each rule produces a flag with a severity and a plain-English reason that quotes the patient's actual values, so that a doctor reading the flag can see immediately why it was raised.

Risk Flag	Condition
Poor sleep	Sleep below 6 hours per day
Sedentary lifestyle	Weekly exercise below 75 minutes, or sitting above 8 hours per day
High stress	Stress level reported as high
Unhealthy diet	Diet reported as frequent fast food, high sugar or irregular
Low water intake	Water intake below 1.5 litres per day
Excess water intake	Water intake above 4 litres per day
Tobacco use	Smoking reported as occasional or daily
Alcohol risk	Alcohol reported as weekly or daily
Overweight / Obese (BMI)	BMI at or above 23 (overweight) or 25 (obese), using Asian-Indian cutoffs
Possible hypertension risk	Two or more of: high stress, sedentary lifestyle, unhealthy diet, tobacco use, alcohol risk, BMI at or above 25

Composite flags such as possible obesity risk and possible hypertension risk are labelled as being for doctor review, and are never phrased as a diagnosis.

4.7 Safety Design

Emergency detection runs before any lifestyle processing on every patient message. Seven red-flag categories are screened: chest pain, severe breathlessness, fainting, stroke signs such as facial droop or one-sided weakness or slurred speech, blood in vomit or stool, thoughts of self-harm, and high fever with neck stiffness. Detection uses both the extraction step and an independent keyword scan of the translated text, so that a failure in one path does not silently disable the check.

When a red flag fires, the consultation status changes to escalated, a high-severity risk flag is recorded, the questionnaire stops, and the patient is told calmly in their own language that these symptoms need prompt in-person medical attention. No wellness score and no lifestyle recommendations are produced for an escalated consultation, because producing lifestyle advice in that situation would be inappropriate.

5. IMPLEMENTATION

5.1 Technology Stack

Layer	Technology	Purpose
Backend framework	Django 5, Django REST Framework	REST API, ORM, admin interface
Authentication	djangorestframework-simplejwt	Token-based authentication with roles
Database	SQLite 3	Consultations, messages, clinical records
Speech and language	Hosted inference API	Speech-to-text, translation, structured extraction
Medical NLP	scispaCy (en_ner_bc5cdr_md)	Symptom and disease entity recognition
Vector store	ChromaDB	Guideline chunk storage and similarity search
PDF generation	WeasyPrint	Clinical report rendering with embedded Indic fonts
Frontend	Next.js (App Router), JavaScript	Patient and doctor interfaces
Styling	Tailwind CSS	Responsive layout and design system
Charts	Recharts	Wellness score gauge and factor breakdown
Deployment	Nginx, Gunicorn, systemd, GitHub Actions	Hosting and continuous deployment
Testing	pytest, pytest-django	Automated test suite (300 tests)

5.2 Implementation of Key Modules

5.2.1 Lifestyle Slot Extraction

Patient answers arrive as free text. The extraction step converts them into typed fields with evidence. The prompt requests a strict JSON structure that is validated against a schema; a failed validation triggers one retry with the validation error appended, after which the turn falls back gracefully rather than crashing.

```
class SlotUpdate(BaseModel):
    slot: str
    value: Any
    quote: str          # the patient's own words
    confidence: float   # 0.0 to 1.0

class ExtractionResult(BaseModel):
    updates: list[SlotUpdate] = []
    symptoms: list[SymptomItem] = []
    red_flags: list[str] = []
    needs_clarification: str | None = None

MIN_CONFIDENCE = 0.6

def extract(consultation, message):
    result = groq_client.chat_json(
        system=EXTRACTION_PROMPT,
        messages=recent_turns(consultation),
        schema=ExtractionResult,
        temperature=0,
    )
    accepted = []
    for update in result.updates:
        if update.confidence < MIN_CONFIDENCE:
            result.needs_clarification = update.slot
            continue
        # cross-check numeric values against a regex parse
        regex_value = regex_numbers.parse(update.slot, message.text_english)
        if regex_value is not None and diverges(regex_value, update.value):
            update.value = regex_value   # prefer the literal number
            logger.warning('extraction.conflict', extra={'slot': update.slot})
        accepted.append(update)
    result.updates = accepted
    return result
```

5.2.2 Wellness Score Calculation

The scoring module is plain Python with no external dependency. The weight table is declared at the top of the file so that it can be read and audited without following any logic.

```
WEIGHTS = {
    'sleep': 15, 'exercise': 15, 'diet': 15, 'stress': 15,
    'smoking': 15, 'alcohol': 10, 'sitting': 10, 'water': 5,
}
BANDS = [(80, 'Good'), (60, 'Fair'), (40, 'Needs attention'), (0, 'Poor')]
MIN_FILLED_SLOTS = 4

def _sleep_sub_score(hours):
    if 7 <= hours <= 9: return 1.0
    if 6 <= hours < 7 or 9 < hours <= 10: return 0.6
    if 5 <= hours < 6: return 0.3
    return 0.0

def _water_sub_score(litres):
    if 2 <= litres <= 3.5: return 1.0
    if 1.5 <= litres < 2: return 0.6
    if 1 <= litres < 1.5: return 0.3
    if litres > 4: return 0.5      # excess is not healthy
    return 0.0

def compute_wellness_score(profile):
    breakdown, earned, available = [], 0.0, 0
    for factor, weight in WEIGHTS.items():
        value = resolve(profile, factor)        # doctor override wins
        if value is None:
            breakdown.append({'factor': factor, 'value': None,
                              'weight': 0, 'points': None,
                              'reason': 'not reported'})
            continue
        sub = SUB_SCORES[factor](value)
        earned += sub * weight
        available += weight
        breakdown.append({'factor': factor, 'value': value,
                          'weight': weight, 'points': round(sub * weight, 1)})
    if filled_slot_count(profile) < MIN_FILLED_SLOTS:
        return None          # too little data to score honestly
    total = round(earned * 100 / available)
    band = next(name for floor, name in BANDS if total >= floor)
    return {'total': total, 'band': band, 'breakdown': breakdown}
```

5.2.3 Risk Rule Evaluation

Risk rules are evaluated after scoring. Each rule fires only on data that is actually present, and every flag carries a reason built from the patient's own values.

```
OVERWEIGHT_BMI, OBESE_BMI = 23.0, 25.0      # Asian-Indian cutoffs

def compute_bmi(patient):
    if not patient.height_cm or not patient.weight_kg:
        return None
    metres = patient.height_cm / 100
    return round(patient.weight_kg / (metres * metres), 1)

def evaluate(profile, patient):
    flags = []
    if profile.sleep_hours is not None and profile.sleep_hours < 6:
        flags.append(RiskFlag(
            code='poor_sleep', label='Poor sleep', severity='medium',
            reason=f'Reports {profile.sleep_hours} hours of sleep a night, '
                   f'below the recommended minimum of 6 hours.'))
    bmi = compute_bmi(patient)
    if bmi is not None and bmi >= OBESE_BMI:
        flags.append(RiskFlag(
            code='obese', label='Obese (BMI)', severity='medium',
```

```
            reason=f'BMI of {bmi} is at or above the Asian-Indian '
                   f'obese cutoff of {OBESE_BMI}.'))
    contributors = count_hypertension_contributors(profile, bmi)
    if contributors >= 2:
        flags.append(RiskFlag(
            code='hypertension_risk',
            label='Possible hypertension risk', severity='high',
            reason=f'Possible risk, for doctor review: '
                   f'{contributors} contributing factors present.'))
    return flags
```

5.2.4 Emergency Escalation

The red-flag check runs before extraction on every message, using both the model's own detection and an independent keyword scan so that a single failure cannot disable the safeguard.

```
RED_FLAGS = {
    'chest_pain': ['chest pain', 'chest tightness', 'pressure in chest'],
    'severe_breathlessness': ['cannot breathe', 'gasping', 'breathless at rest'],
    'stroke_signs': ['face drooping', 'slurred speech', 'one side weak'],
    'fainting': ['fainted', 'passed out', 'blacked out'],
    # ... blood_in_vomit_or_stool, self_harm_thoughts, high_fever_stiff_neck
}

def detect(message_english, extraction):
    found = set(extraction.red_flags)
    lowered = message_english.lower()
    for code, phrases in RED_FLAGS.items():
        if any(phrase in lowered for phrase in phrases):
            found.add(code)
    return found

def trigger(consultation, codes):
    consultation.status = Consultation.ESCALATED
    consultation.save(update_fields=['status'])
    for code in codes:
        RiskFlag.objects.create(
            consultation=consultation, code=code,
            label=RED_FLAG_LABELS[code], severity='high',
            reason='Reported during consultation; requires prompt review.')
    # run_analysis() is deliberately NOT called for an escalated
    # consultation - no wellness score, no lifestyle recommendations
    return escalation_message(consultation.language)
```

5.2.5 Guideline Retrieval

Guideline PDFs are split into overlapping chunks, tagged with a topic and an age group, and stored with their embeddings. At query time, retrieval is filtered by the patient's age group and a distance threshold is applied so that an unrelated passage is never used as a citation.

```
MAX_RELEVANT_DISTANCE = 1.3

def retrieve(query, k=4, topic=None, age_group='adults'):
    collection = get_existing_collection('guidelines')
    if collection is None:
        logger.warning('retriever.no_collection')
        return []
    where = {'$and': [{'age_group': {'$in': [age_group, 'unknown']}}]}
    if topic:
        where['$and'].append({'topic': topic})
    result = collection.query(query_texts=[query], n_results=k, where=where)
    chunks = []
     for text, meta, distance in zip(result['documents'][0],
                                     result['metadatas'][0],
                                    result['distances'][0]):
        if distance > MAX_RELEVANT_DISTANCE:
            continue     # Chroma always returns k results; drop the far ones
        chunks.append({'text': text, 'file': meta['file'],
                       'page': meta['page'], 'distance': distance})
    if not chunks:
```

```
        logger.info('retriever.no_relevant_chunks', extra={'query': query})
    return chunks
```

5.2.6 Doctor Override and Re-scoring

When a doctor corrects a value, the correction is stored separately from the extracted value so that the original record of what the patient said is never overwritten. The score and risk flags are then recomputed from the corrected data.

```
class DoctorLifestyleOverrideView(APIView):
    permission_classes = [IsDoctor]

    def patch(self, request, pk):
        consultation = self.get_assigned_consultation(request.user, pk)
        profile = consultation.lifestyle_profile
        overrides = dict(profile.doctor_overrides or {})
        for field, value in request.data.items():
            if field not in LIFESTYLE_FIELDS:
                raise ValidationError(f'Unknown field: {field}')
            overrides[field] = value
        profile.doctor_overrides = overrides      # extracted values untouched
        profile.save(update_fields=['doctor_overrides'])
        pipeline.rescore_only(consultation)       # score + flags, not the NLP
        return Response(DoctorConsultationDetailSerializer(consultation).data)
```

5.3 Output Screenshots

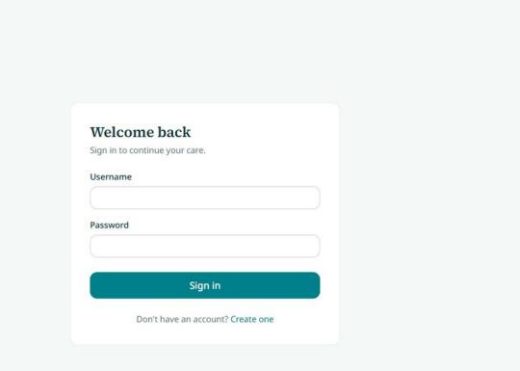


Figure 5.1 - Login screen. Patients and doctors use the same entry point and are routed by role.

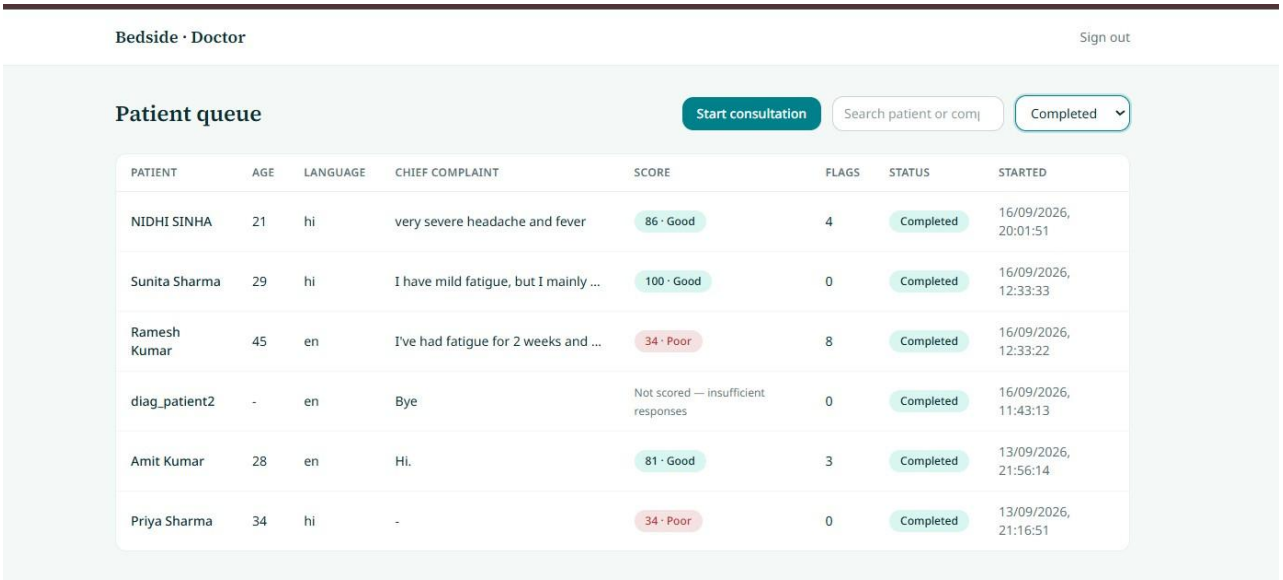


Figure 5.2 - Doctor's patient queue, showing the wellness score band, the number of risk flags and the consultation status for each patient. The fourth row demonstrates the insufficient-data rule: a consultation with too few answers is shown as "Not scored" rather than given a misleading number.

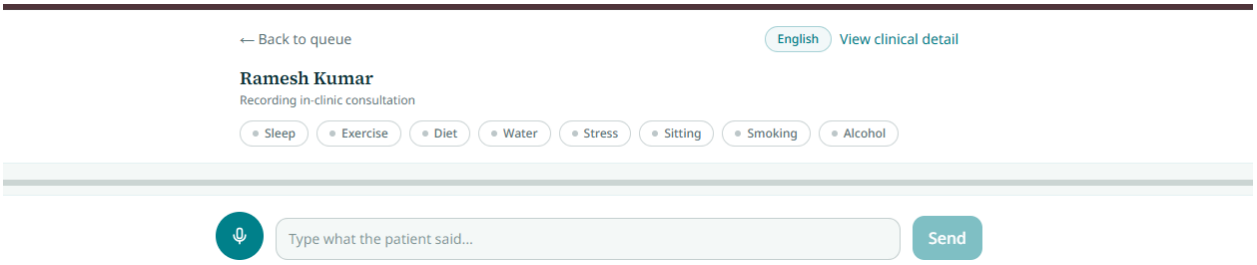


Figure 5.3 - Doctor-led consultation recording, English. The progress chips at the top show which of the eight lifestyle areas have been covered.

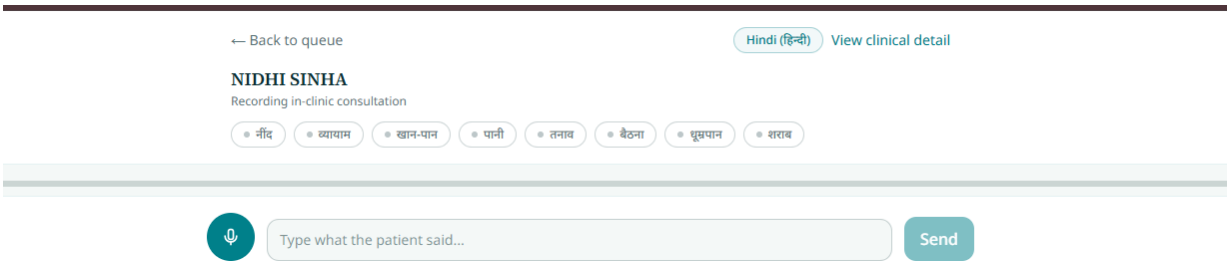


Figure 5.4 - The same screen for a Hindi-speaking patient. The interface, the progress chips and the assistant's questions all switch to the patient's language.

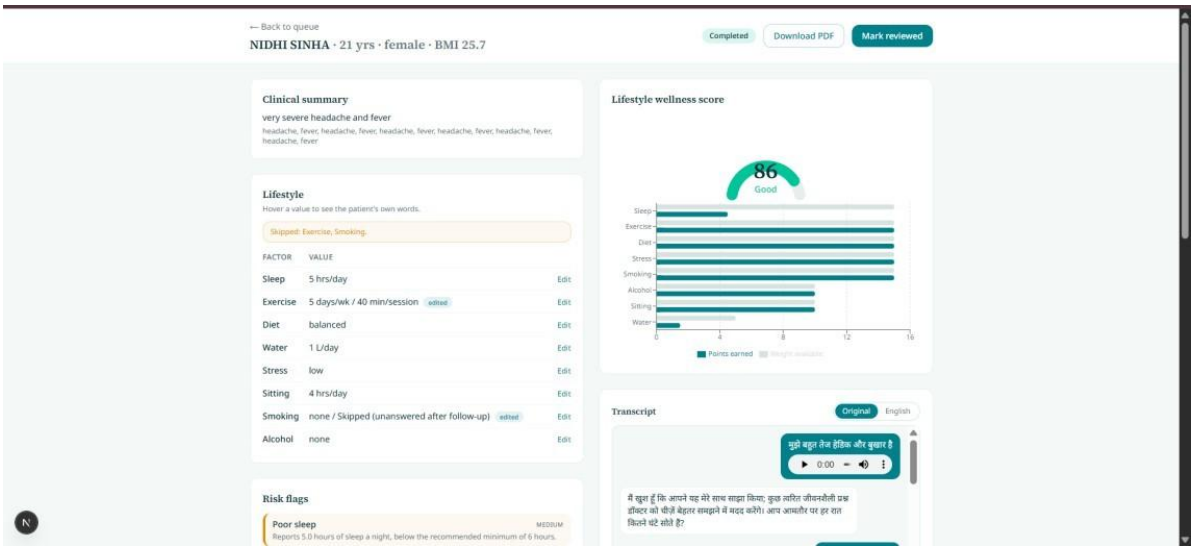


Figure 5.5 - Doctor's clinical detail view. The lifestyle table shows each extracted value with an edit control, and marks values the doctor has already corrected. The gauge shows the wellness score and band; the bar chart shows points earned against weight available for each factor. The transcript panel plays back the original audio and toggles between the patient's language and English.

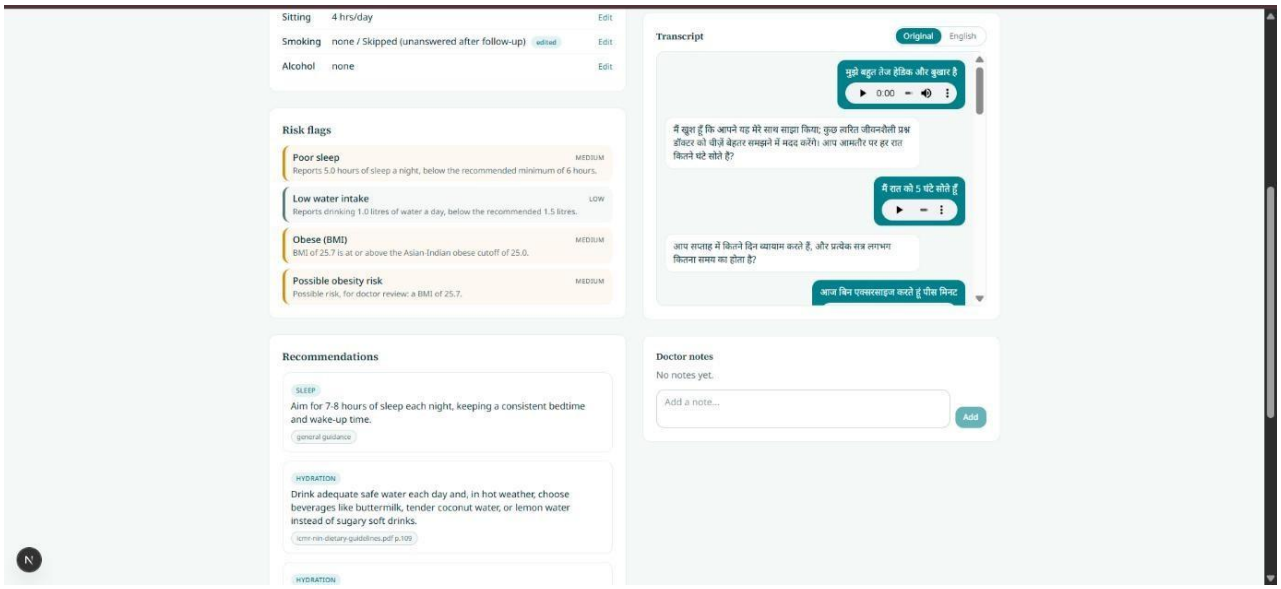


Figure 5.6 - Risk flags with their reasons, and guideline-grounded recommendations. Each recommendation carries the document and page it was retrieved from; where no relevant passage was found, the source is shown as general guidance rather than a fabricated citation.

6. RESULTS AND CONCLUSION

6.1 Testing and Verification

The system was verified through an automated test suite and through end-to-end manual testing in a browser with real voice input. The test suite runs entirely offline in a deterministic mode, so that results do not depend on the availability or behaviour of the external inference service.

Area Tested	Coverage
Authentication and roles	Registration, login, consent enforcement, role-based access
Consultation endpoints	Message posting, state retrieval, ownership checks, error handling
Conversation engine	Slot filling, clarification limit, turn cap, escalation path, multilingual flow
Scoring	Exact score for a reference patient, weight rescaling, bounded upper limits, insufficient-data rule, doctor overrides
Risk rules	Each individual rule, BMI edge cases, composite rule thresholds
Retrieval	Chunking, ingestion, age-group filtering, relevance threshold, fallback behaviour
Doctor endpoints	Queue ordering and filtering, override and re-scoring, notes, review sign-off, cross-patient access denial
Reporting	PDF generation, incomplete-consultation warning, escalated-consultation wording
Total	**300 automated tests, all passing**

6.2 Sample Case Study

The following case was used as the reference test throughout development. A 45-year-old patient reported fatigue for two weeks with mild headaches and no fever, and gave the lifestyle answers shown below.

Lifestyle Area	Reported Value	Sub-score
Sleep	5 hours per day	0.3
Exercise	None	0.0
Diet	Frequent fast food	0.2
Water intake	1 litre per day	0.3
Stress	High	0.0
Sitting	9 hours per day	0.3
Smoking	None	1.0
Alcohol	Occasional	0.7

The computed Lifestyle Wellness Score was 34 out of 100, placing the patient in the Poor band. Eight risk flags were raised: poor sleep, sedentary lifestyle, high stress, unhealthy diet, low water intake, overweight and obese by BMI, and possible hypertension risk. The recommendations returned were grounded in the ICMR-NIN dietary guidelines and the WHO physical activity guidelines, each carrying its page number. The same inputs produce the same score on every run, and this case is asserted as an exact value in the test suite.

6.3 Observations

- **Separating understanding from judgement worked.** Using the language model only for extraction, and plain Python for every clinical calculation, produced a system whose output can be explained completely. The score can be recomputed by hand from the weight table.
- **Honest failure is better than confident error.** Two cases made this concrete during development. Early versions scored a consultation with only two answers as 100 out of 100 because weight rescaling stretched two favourable answers across the whole scale; the fix was to refuse to score below four answers. Separately, a sleep recommendation retrieved from the children's section of the WHO guidelines was being served to an adult; the fix was to tag every chunk with an age group and filter retrieval by it.
- **Retrieval needs a relevance threshold.** A vector database returns the requested number of nearest results whether or not any of them are relevant. Without a distance cutoff, a query about sleep was returning exercise guidance and citing it. Measured distances for genuinely relevant passages were in the 0.3 to 0.7 range against 1.6 to 2.0 for unrelated ones, which made the threshold straightforward to set.
- **The doctor must remain in control.** Extraction from conversational speech is imperfect. Making every value editable, with live re-scoring, converts an occasional extraction error from a clinical risk into a small correction.

- **Multilingual support is not only translation.** Rendering Devanagari and other Indic scripts correctly in the PDF report required embedding the fonts in the document itself; relying on the reader's system fonts produced empty boxes.

6.4 Conclusion

This project set out to capture lifestyle information that is clinically important but routinely lost, and to do so in the patient's own language without consuming the doctor's limited consultation time. The finished system does this end to end. A patient speaks naturally in one of nine languages; the system extracts eight structured lifestyle fields with the patient's own words attached as evidence; it computes a reproducible wellness score and a set of explained risk flags; it retrieves guideline-backed recommendations with page-level citations; and it presents the whole record to the doctor as something to review and correct rather than something to accept.

The design decision that shaped the project most was restricting the language model to the task it is genuinely good at, which is understanding what a person said, and keeping every clinical calculation in ordinary, auditable code. This is what allows the system's central number to be explained to a doctor, to an examiner, or to the patient, without appealing to the opacity of a model.

Equally important was designing for the cases where the system does not know. A consultation with too few answers produces no score. A retrieval that finds nothing relevant produces general guidance labelled as such rather than a fabricated citation. A patient reporting chest pain is sent to a doctor immediately rather than asked how many hours they sleep. In a clinical setting, these refusals are as much a feature as the outputs.

The system remains a screening and documentation aid. It is not clinically validated and does not diagnose. Within those limits, it demonstrates that a structured, multilingual, guideline-grounded lifestyle assessment can be collected reliably through conversation and delivered to a doctor in a form that is immediately usable.

7. FUTURE SCOPE

The current system establishes the conversational assessment pipeline and the clinical review workflow. Several extensions follow naturally from it.

7.1 Wearable Device Integration

Sleep duration, step count, resting heart rate and activity minutes are already measured continuously by consumer fitness bands and smartwatches. Feeding these measurements directly into the wellness score would replace self-reported values with objective ones for the areas where self-report is least reliable, while leaving diet, stress and substance use to the conversation.

7.2 Longitudinal Progress Tracking

At present each consultation is scored independently. Storing scores over time and presenting a trend line would let a doctor see whether advice given at a previous visit produced any change, which is more clinically useful than a single reading.

7.3 AI-Assisted Nutrition Planning

The system currently identifies an unhealthy diet and recommends general dietary changes. A natural extension is a structured meal plan built from regional Indian foods, matched to the patient's stated eating pattern, budget and any conditions the doctor records.

7.4 Mental Health Assessment Module

Stress is currently captured as a single three-level field. A dedicated module using a validated screening instrument, with the same conversational and multilingual interface, would produce a more meaningful assessment, subject to careful design of the escalation path for distress detected during screening.

7.5 Medication Adherence Reminders

Where a doctor has prescribed treatment, the system's existing multilingual voice interface could deliver adherence reminders and record the patient's spoken response, closing the loop between consultation and follow-up.

7.6 Voice Emotion Analysis

The system already receives the patient's audio. Analysing prosodic features alongside the transcript could surface signs of distress or pain that the words alone do not convey, presented to the doctor as an observation rather than a conclusion.

7.7 Hospital Information System Integration

Integration with hospital management and electronic medical record systems through a standard interface such as HL7 FHIR would place the lifestyle assessment directly in the patient's existing medical record rather than in a separate application.

7.8 Expanded Language Coverage

Nine languages are supported at present. Extending coverage to the remaining scheduled languages, and improving recognition of code-mixed speech in which patients move between a regional language and English within a single sentence, would widen the system's reach considerably.

7.9 Clinical Validation Study

The most significant next step is not technical. Establishing whether the Lifestyle Wellness Score correlates with clinical outcomes would require a prospective study comparing scores against measured indicators over time. Until such a study is carried out, the score remains a documentation aid rather than a clinical instrument.

8. REFERENCES

[1] World Health Organization, WHO Guidelines on Physical Activity and Sedentary Behaviour, World Health Organization, Geneva, 2020.

[2] National Institute of Nutrition, Indian Council of Medical Research, Dietary Guidelines for Indians, ICMR-NIN, Hyderabad, 2024.

[3] World Health Organization Expert Consultation, "Appropriate body-mass index for Asian populations and its implications for policy and intervention strategies," The Lancet, vol. 363, no. 9403, pp. 157-163, 2004.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," Proceedings of the 40th International Conference on Machine Learning, 2023.

[5] M. Neumann, D. King, I. Beltagy and W. Ammar, "ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing," Proceedings of the 18th BioNLP Workshop, Association for Computational Linguistics, pp. 319-327, 2019.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 3982-3992, 2019.

[8] Django Software Foundation, Django Documentation, Version 5.0. [Online]. Available: https://docs.djangoproject.com/

[9] Encode OSS Ltd., Django REST Framework Documentation. [Online]. Available: https://www.django-rest-framework.org/

[10] Vercel Inc., Next.js Documentation, App Router. [Online]. Available: https://nextjs.org/docs

[11] Chroma, ChromaDB Documentation. [Online]. Available: https://docs.trychroma.com/

[12] CourtBouillon, WeasyPrint Documentation. [Online]. Available: https://doc.courtbouillon.org/weasyprint/

[13] Mozilla Developer Network, MediaStream Recording API. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaStream_Recording_API

[14] Indian Council of Medical Research, National Ethical Guidelines for Biomedical and Health Research Involving Human Participants, ICMR, New Delhi, 2017.